On-Device Voice-Controlled UGV Using MFCC and DTW

Divyansh Maheshwari, Shriyansh Chawda and GVV Sharma
Department of Electrical Engineering
Indian Institute of Technology, Hyderabad
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—This paper presents a cloud-free voice-controlled Unmanned Ground Vehicle (UGV) that performs all speech recognition on an Android smartphone. The phone records 1.5 s of audio, extracts 26-dimensional Mel-Frequency Cepstral Coefficient (MFCC) features, and applies Dynamic Time Warping (DTW) with Best-of-N template matching (up to $N = 10$ templates per command) to identify one of five commands: *forward*, *back*, *left*, *right*, and *stop*. The recognized command is sent as a 7-byte string over Bluetooth Low Energy (BLE) to an ESP32, which drives a differential-drive chassis via an L293D motor driver. Moving recognition off the ESP32 reduces latency from 2–4 s to 1.6 s and frees 80 kB of flash. User adaptation is supported through up to ten recorded templates per command, achieving 90–95% accuracy across 10 speakers and demonstrating language-agnostic operation in English, Hindi, Gujarati, and Tamil.**

***Index Terms*—UGV, voice control, MFCC, DTW, ESP32, BLE, Android, speech recognition, embedded systems**

I. Introduction

Voice-based control of robotic platforms provides an intuitive human–machine interface, eliminating the need for dedicated hardware controllers. Prior work on speech-driven robots has largely relied on cloud-based automatic speech recognition (ASR) services, which introduce network latency, require continuous internet connectivity, and raise data-privacy concerns [1]. On the other hand, running full ASR models directly on low-power microcontrollers has been constrained by limited memory and compute resources.

Template-based approaches such as Dynamic Time Warping offer a practical middle ground: they are computationally light, require no training data beyond a small set of recorded templates, and run entirely on-device. DTW has been widely used in isolated word recognition [2], and MFCC features are the standard front-end for speech systems [3]. Prior work on embedded keyword spotting has shown that neural approaches can be deployed on microcontrollers [4], [5], but require substantially more training data and memory than the template-based method presented here. BLE has been established as an effective low-power communication link for small robotic platforms [6].

This work places MFCC extraction and DTW matching on an Android smartphone, exploiting its significantly more powerful processor (compared to the ESP32) and microphone, while the ESP32 microcontroller is reserved for motor control and BLE communication. The contribution of this paper is threefold:

1) A fully on-device, cloud-free voice recognition pipeline running on a commodity Android phone.
2) A Best-of-N DTW strategy enabling user adaptation with up to ten templates per command.
3) A latency-optimized BLE communication scheme transmitting only a 7-byte command string instead of raw audio frames, reducing the BLE payload by over 99%.

II. System Architecture

A. Overview

The system comprises two hardware nodes connected via BLE: an Android smartphone acting as the sensing and recognition unit, and an ESP32 microcontroller acting as the motion controller.

Initially, MFCC extraction and DTW matching were executed directly on the ESP32. However, this consumed ∼80 kB of flash, required transferring 24,000 bytes of raw audio over BLE before processing, and introduced 2–4 s of latency. Migrating the recognition pipeline to the Android phone reduces the BLE payload to a 7-byte command string and end-to-end latency to ∼1.6 s, while the ESP32 is reserved solely for BLE reception and motor control.

TABLE I
Architecture Comparison: Old vs. New Pipeline

Stage	Old	New
Audio capture	ESP32 mic	Android mic
MFCC extraction	ESP32 (C++)	Android (Java)
DTW matching	ESP32 (C++)	Android (Java)
BLE payload	24,000 bytes	7 bytes
ESP32 flash	∼80 kB extra	Minimal
End-to-end latency	2–4 s	∼1.6 s

B. Data Flow

The complete pipeline proceeds as follows:

1) The Android app records 1.5 s of audio at 8 kHz mono (12,000 samples, 16-bit PCM) on button press.
2) `MFCCProcessor` extracts a 92×26 feature matrix (13 raw + 13 delta MFCC coefficients per frame).

3) `DTWMatcher` computes the Best-of-N DTW score against all stored templates for each of the five commands.
4) If the minimum score is below the acceptance threshold ($\tau = 12.0$), the command string is sent over BLE.
5) The ESP32 receives the string and actuates the motors via the L293D driver.

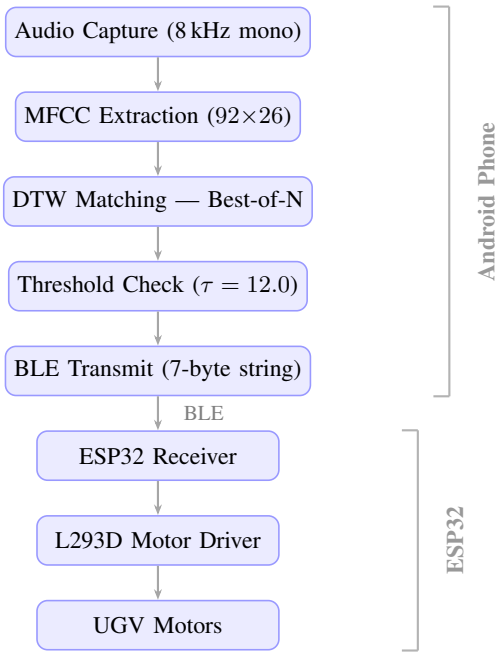

Fig. 1. System pipeline block diagram.

III. Hardware Setup

A. Components

TABLE II
List of Components

Component	Qty.	Description
Chassis frame	1	Holds all components
Side wheels (DC motors)	2	Drive wheels
Caster wheel	1	Stability and smooth turning
ESP32 dev board	1	Microcontroller + BLE
L293D motor driver IC	1	Drives DC motors
DC motors (TT gear)	2	Propulsion
Breadboard	1	Circuit connections
Jumper wires	11	Electrical connections
Power bank / LiPo	1	Portable power supply
Android phone (API 23+)	1	Recognition + BLE central
Micro-USB / USB-C cable	1	ESP32 flashing

B. Motor Wiring

The ESP32 controls the L293D via four digital GPIO pins. The enable pins EN1 and EN2 of the L293D are wired permanently HIGH.

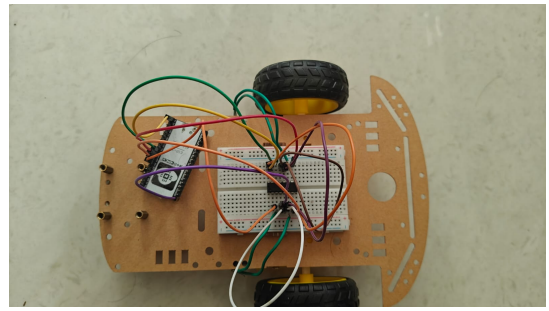

Fig. 2. Top-down view of the assembled UGV prototype, detailing the ESP32, breadboard, L293D motor driver, and chassis wiring.

TABLE III
ESP32 to L293D Connections

ESP32 GPIO	L293D Pin	Function
GPIO 32	IN1	Motor A — direction 1
GPIO 33	IN2	Motor A — direction 2
GPIO 25	IN3	Motor B — direction 1
GPIO 26	IN4	Motor B — direction 2
GND	GND	Common ground
3.3 V	EN1, EN2	Motor channel enable (HIGH)

C. Full Schematic

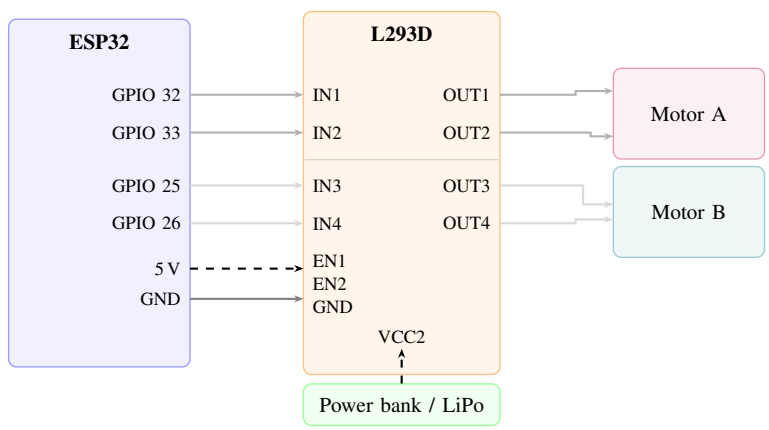

Fig. 3. Wiring schematic: ESP32 to L293D to DC motors. Dark arrows — Motor A channel (GPIO 32/33 → IN1/IN2 → OUT1/OUT2). Light arrows — Motor B channel (GPIO 25/26 → IN3/IN4 → OUT3/OUT4). Dashed — power connections (3.3 V to EN1, EN2; motor supply to VCC2).

D. Motor Truth Table

TABLE IV
Motor Direction Truth Table

IN1	IN2	IN3	IN4	Result
H	L	H	L	Move Forward
L	H	L	H	Move Back
L	H	H	L	Turn Left
H	L	L	H	Turn Right
L	L	L	L	Motor Stop

IV. Signal Processing — MFCC Feature Extraction

MFCC extraction transforms raw audio into a compact, speaker-adapted feature representation. The pipeline follows seven sequential stages.

A. Framing Parameters

Audio is segmented into overlapping frames:

- Frame size: $N = 256$ samples (32 ms at 8 kHz)
- Hop size: $H = 128$ samples (16 ms, 50% overlap)
- Total frames for 1.5 s: $(12000 - 256)/128 + 1 = 92$

B. Pre-emphasis

A first-order high-pass filter boosts high-frequency content:

$$y[n] = x[n] - \alpha\, x[n-1], \quad \alpha = 0.97 \tag{1}$$

The signal is normalized to $[-1, 1]$ by dividing by 32768 prior to windowing.

C. Hamming Window

Each frame is multiplied by a Hamming window to reduce spectral leakage:

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), \quad n = 0, \ldots, N-1 \tag{2}$$

D. Fast Fourier Transform and Power Spectrum

A 256-point radix-2 Cooley–Tukey FFT is applied. The one-sided power spectrum is:

$$P[k] = \frac{|X[k]|^2}{N}, \quad k = 0, \ldots, \frac{N}{2} \tag{3}$$

where $X[k]$ denotes the FFT output. Only the first $N/2+1 = 129$ bins are retained.

E. Mel Filterbank

The Mel scale maps frequency to a perceptually uniform scale:

$$m = 2595\log_{10}\left(1 + \frac{f}{700}\right) \tag{4}$$

Twenty-six triangular filters equally spaced on the Mel scale are applied to the power spectrum. Log energy is computed per filter:

$$E[m] = \log\left(\sum_k P[k]\, H_m[k]\right), \quad m = 0, \ldots, 25 \tag{5}$$

where $H_m[k]$ is the m-th triangular filter. A floor of 10^{-10} prevents $\log(0)$.

F. Discrete Cosine Transform

DCT-II with orthonormal normalization decorrelates the 26 log-energies into 13 MFCC coefficients:

$$c[k] = w_k \sum_{n=0}^{M-1} E[n]\cos\left(\frac{\pi k(2n+1)}{2M}\right), \quad k = 0, \ldots, 12 \tag{6}$$

$$w_k = \begin{cases} \sqrt{1/M} & k = 0 \\ \sqrt{2/M} & k > 0 \end{cases} \tag{7}$$

where $M = 26$.

G. Delta Coefficients

Delta (velocity) features capture temporal dynamics and are appended to each frame, doubling the feature dimension from 13 to 26:

$$\Delta c[f][k] = \frac{c[f+1][k] - c[f-1][k]}{2} \tag{8}$$

Edge frames use the nearest valid neighbor.

H. Cepstral Mean and Variance Normalisation (CMVN)

CMVN removes channel effects and normalizes each coefficient across all frames F:

$$\mu_k = \frac{1}{F}\sum_{f=1}^{F} c[f][k] \tag{9}$$

$$\sigma_k = \sqrt{\frac{1}{F}\sum_{f=1}^{F}\big(c[f][k] - \mu_k\big)^2} \tag{10}$$

$$\hat{c}[f][k] = \frac{c[f][k] - \mu_k}{\sigma_k + \varepsilon} \tag{11}$$

where $\varepsilon = 10^{-8}$ prevents division by zero.

The final MFCC output is a matrix of shape 92×26, representing 92 frames each with 26 normalized coefficients.

V. Pattern Matching — Dynamic Time Warping

A. DTW Algorithm

Given a query sequence $\mathbf{Q}$ of length F_1 and a template $\mathbf{T}$ of length F_2, each with 26-dimensional feature vectors, DTW finds the optimal non-linear alignment. The local distance is the Euclidean norm:

$$d(i,j) = \|\mathbf{Q}[i] - \mathbf{T}[j]\|_2 \tag{12}$$

The accumulated cost matrix is filled by the recurrence:

$$D[i][j] = d(i,j) + \min\big(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]\big) \tag{13}$$

A parallel path-length matrix L tracks the number of steps taken along the warping path:

$$L[i][j] = 1 + L[p^*] \tag{14}$$

where $p^* = \arg\min\big(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]\big)$ denotes the predecessor cell selected by the DTW recurrence.

The normalized DTW score is:

$$\text{score} = \frac{D[F_1-1][F_2-1]}{L[F_1-1][F_2-1]} \tag{15}$$

This normalization removes the bias introduced by different word durations. Memory usage is reduced from $O(F_1F_2)$ to $O(F_2)$ by maintaining only two rolling rows of D and L.

Algorithm 1 MFCC Feature Extraction

Require: Raw audio samples $x[n]$, $n = 0, \ldots, 11999$ (1.5 s at 8 kHz)
Ensure: Feature matrix $\hat{\mathbf{C}} \in \mathbb{R}^{92\times 26}$

Pre-emphasis: $y[n] \leftarrow x[n] - 0.97\,x[n-1]$; normalize by 32768
for each frame $f = 0$ **to** 91 **do**
  Extract samples $y[fH : fH + N]$ with $N = 256$, $H = 128$
  Apply Hamming window: $s[n] \leftarrow y[n] \cdot w[n]$
  Compute 256-point FFT; form power spectrum $P[k] = |X[k]|^2/N$
  Apply 26-band Mel filterbank; compute log energies $E[m]$
  Apply DCT-II to $E[m]$; retain first 13 coefficients $\mathbf{c}[f]$
end for
for each frame $f = 0$ **to** 91 **do**
  Compute delta: $\Delta c[f][k] \leftarrow (c[f+1][k] - c[f-1][k])/2$
  Concatenate: $\mathbf{c}[f] \leftarrow [\mathbf{c}[f] \parallel \Delta\mathbf{c}[f]]$ (26-dim)
end for
Compute per-coefficient mean μ_k and std σ_k across all frames
CMVN: $\hat{c}[f][k] \leftarrow (c[f][k] - \mu_k)/(\sigma_k + 10^{-8})$
return $\hat{\mathbf{C}}$

B. Best-of-N Strategy

Each command stores up to $N = 10$ templates (index 0 from the original trained header, indices 1–9 recorded in-app). The command score is the minimum DTW score across all stored templates:

$$s(\text{cmd}) = \min_{i=0}^{N-1} \text{score}(\mathbf{Q}, \mathbf{T}_i) \tag{16}$$

Recognition proceeds by selecting the command with the lowest score, subject to an acceptance threshold τ:

$$\hat{c} = \arg\min_{\text{cmd}} s(\text{cmd}), \quad \text{if } \min_{\text{cmd}} s(\text{cmd}) < \tau \tag{17}$$

The default threshold is $\tau = 12.0$. Table VI summarises the interpretation of DTW scores observed in practice.

VI. Recognition Algorithm

This section presents the complete end-to-end voice recognition pipeline as formal algorithms. Algorithm 1 describes MFCC feature extraction, and Algorithm 2 describes the DTW-based command recognition with Best-of-N template matching.

VII. Software Implementation

A. ESP32 Firmware

The firmware is written in C++ using the Arduino framework and built with PlatformIO. It contains three source files.

ble_receive.cpp implements a BLE GATT server with two characteristics: a WRITE characteristic (`UUID: ...9002`) for receiving command strings, and a NOTIFY characteristic (`UUID: ...9003`) for sending acknowledgements. Only the five known command strings are accepted; all other writes are silently discarded. On client disconnect, BLE advertising restarts automatically.

motor.cpp drives the L293D via four GPIO pins (32, 33, 25, 26) using simple digital writes. No PWM speed control is used.

main.cpp implements the command dispatcher:

Listing 1. ESP32 command dispatcher (main.cpp)

```
void loop() {
  char cmd[16] = {0};
  if (!ble_receive_command(cmd, sizeof(cmd)))
    { delay(5); return; }

  if      (!strcmp(cmd,"forward")) { motor_forward()
      ; }
  else if (!strcmp(cmd,"back"))    { motor_back(); }
  else if (!strcmp(cmd,"left"))
    { motor_left();  delay(600); motor_stop(); }
  else if (!strcmp(cmd,"right"))
    { motor_right(); delay(600); motor_stop(); }
  else if (!strcmp(cmd,"stop"))    { motor_stop(); }
}
```

forward and *back* run continuously until a *stop* command is received. *left* and *right* execute for 600 ms then auto-stop,

Algorithm 2 Voice Command Recognition via DTW + Best-of-N

Require: Query MFCC matrix $\mathbf{Q} \in \mathbb{R}^{F_1\times 26}$, template sets $\{\mathbf{T}_{c,i}\}$ for each command c, threshold $\tau = 12.0$
Ensure: Recognized command $\hat{c}$, or REJECT

for each command $c \in \{$*forward, back, left, right, stop*$\}$ **do**
  $s_c \leftarrow \infty$
  for each stored template $\mathbf{T}_{c,i}$ of length F_2 **do**
    Initialize $D_{\text{prev}}[j] \leftarrow \infty$, $L_{\text{prev}}[j] \leftarrow 0$ for $j = 0, \ldots, F_2-1$
    for $i = 0$ **to** $F_1 - 1$ **do**
      for $j = 0$ **to** $F_2 - 1$ **do**
        $d \leftarrow \|\mathbf{Q}[i] - \mathbf{T}_{c,i}[j]\|_2$
        $p^* \leftarrow \arg\min(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1])$
        $D_{\text{curr}}[j] \leftarrow d + D_{p^*}$; $L_{\text{curr}}[j] \leftarrow 1 + L_{p^*}$
      end for
      $D_{\text{prev}} \leftarrow D_{\text{curr}}$; $L_{\text{prev}} \leftarrow L_{\text{curr}}$
    end for
    score $\leftarrow D_{\text{prev}}[F_2-1] \,/\, L_{\text{prev}}[F_2-1]$
    $s_c \leftarrow \min(s_c,\ \text{score})$ {Best-of-N}
  end for
end for
$\hat{c} \leftarrow \arg\min_c\ s_c$
if $s_{\hat{c}} < \tau$ **then**
  return $\hat{c}$
else
  return REJECT
end if

producing approximately a 90° turn. The turn duration is tunable to adjust the turning angle.

B. Android Application

The Android app records 1.5 s of audio at 8 kHz mono, extracts MFCC features, and runs DTW matching entirely on-device. All processing runs on a background thread, with results delivered to the UI thread on completion. If the best DTW score falls below the threshold τ, the command string is written to the BLE characteristic. The app also provides a debug screen showing live DTW scores per command and a training interface for recording additional templates. A 200 ms delay before each `AudioRecord` initialisation prevents microphone release errors on rapid consecutive recordings.

C. Template Conversion Script

A one-time Python script (`convert_templates.py`) converts the C header `templates.h` — which stores the original trained MFCC templates as float arrays — into the binary `.bin` format consumed by `TemplateStore`. The script uses a regular expression to locate each template array and its frame-count `#define`, strips the C-style `f` suffix from float literals, and writes the binary file with `struct.pack`.

D. Out-of-the-Box Deployment

A key advantage of this architecture is its straightforward deployment. By directly uploading the provided C++ firmware to the ESP32 and installing the companion APK on an Android device, the system requires minimal configuration. The ESP32 is configured with a fixed MAC address matching the one expected by the application. Therefore, once the user manually pairs the ESP32 with their smartphone via the standard Android Bluetooth settings, the UGV becomes fully operational. The user can then simply open the app, speak a command, and the vehicle will respond promptly (within approximately 1.6 s).

VIII. User Adaptation and Tuning

A. Recording Additional Templates

Template index 0 (loaded from `assets`) is the original trained template. Indices 1–9 are recorded in-app via `DebugActivity`. Table V lists recommended sample counts per scenario.

TABLE V
Recommended Template Count per Scenario

Scenario	Min.	Recommended
Quick prototype, quiet room	1	3–5
Daily driver, single speaker	5	8–10
Noisy environment	8	10
Shared templates across speakers	Not supported	

B. Threshold Tuning

The optimal threshold lies between the highest correct-command score and the lowest wrong-command score observed in the debug view. With 8–10 templates per command, a threshold of 8.0–10.0 is achievable for a single speaker in a quiet environment.

C. Turn Duration

The 600 ms turn duration in `main.cpp` is calibrated to produce approximately 90° on the target chassis at full motor speed. It should be increased for heavier chassis or slower motors, and decreased for faster ones.

IX. Results and Discussion

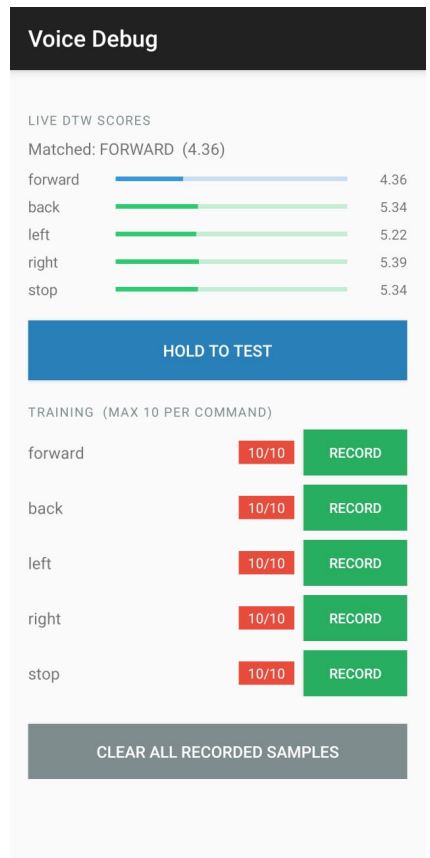

Fig. 4. Debug Activity screen showing live DTW scores after speaking *forward*. The blue bar indicates the winning command (score 4.36), while all other commands score above 5.0. All five commands have 10 recorded templates (10/10), producing well-separated scores and reliable recognition.

A. Multi-Speaker Evaluation

The system was evaluated with **10 speakers** each recording 10 templates per command using the in-app training interface. Testing was conducted in a quiet indoor environment. Overall recognition accuracy across all speakers and commands was **90–95%**, summarised in Table VI.

TABLE VI
TYPICAL DTW SCORES (SINGLE SPEAKER, QUIET ENVIRONMENT)

Condition	Score Range	Outcome
Correct command, 10 templates	3.0 – 7.5	Match ($< \tau$)
Correct command, 1 template	5.0 – 11.0	Match / borderline
Wrong command	11.0 – 22.0	No match ($> \tau$)
Silent / noise	18.0 – 30.0	No match

This result is noteworthy because DTW is inherently a speaker-dependent method — the base templates were recorded by a single speaker, yet the system achieved high accuracy across all 10 participants. It should be noted that 10 speakers represents a limited evaluation cohort; larger-scale testing across more diverse acoustic conditions is planned as future work. This generalization is attributed to two design decisions: CMVN normalization (Equation 11), which removes speaker-dependent channel effects from the feature vectors, and the Best-of-N strategy (Equation 16), which allows each user to personalize recognition with their own recorded templates.

B. Cross-Language Evaluation

The system was also tested with speakers using commands spoken in **Hindi, Gujarati, and Tamil**. Since the recognition pipeline operates entirely on acoustic features (MFCC + DTW) with no linguistic model, it is language-agnostic — any consistent spoken word can serve as a command regardless of the language it comes from. Recognition accuracy in the range of 90–95% was observed across all tested languages, comparable to the English baseline, further validating the generality of the approach.

C. Latency

End-to-end latency was measured at approximately 1.6 s, compared to 2–4 s in the prior architecture where raw audio was streamed to the ESP32 for processing. The reduction is attributed to eliminating the 24,000-byte BLE audio transfer and replacing it with a 7-byte command string.

D. Limitations

The system does not natively support multiple speakers without per-user template recording. Performance degrades in high-noise environments, though Android's built-in `NoiseSuppressor` API partially mitigates this. Future work will explore lightweight neural network classifiers and speaker-independent models deployable on-device.

X. CONCLUSIONS

This paper demonstrated that MFCC feature extraction and DTW pattern matching can be migrated from a resource-constrained ESP32 microcontroller to a commodity Android smartphone, yielding a fully on-device, cloud-free voice control pipeline for a UGV. The architectural shift reduced end-to-end latency from 2–4 s to 1.6 s and freed 80 kB of microcontroller flash, while the Best-of-N strategy and CMVN normalization together enabled 90–95% recognition accuracy across 10 speakers and four languages (English, Hindi, Gujarati, and Tamil) despite DTW being inherently speaker-dependent.

The current system has two notable limitations: per-user template recording is required for new speakers, and performance degrades in high-noise environments. Future work will address these by exploring lightweight convolutional or recurrent keyword-spotting models deployable on-device, incorporating data-augmentation to improve noise robustness, and conducting a larger-scale evaluation across a broader and more diverse speaker population to provide statistically robust accuracy estimates.

REFERENCES

[1] G. Li, S. Mei, H. Yang, X. Zhang, H. Wang, and F. Sun, "Speech interaction with robots: A review," *International Journal of Advanced Robotic Systems*, vol. 14, no. 6, 2017.

[2] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.

[3] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[4] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," in *arXiv preprint arXiv:1711.07128*, 2017.

[5] S. Choi, S. Seo, and B. Moon, "An efficient keyword spotting technique using a recurrent neural network for resource-constrained devices," *Applied Sciences*, vol. 9, no. 9, p. 1885, 2019.

[6] K. Mikhaylov and J. Tervonen, "Evaluation of power consumption for Bluetooth Low Energy," in *2013 IEEE 24th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC)*. IEEE, 2013, pp. 155–159.

[7] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[8] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[9] A. Maier *et al.*, "Design and implementation of ESP32-based IoT devices," *Sensors*, vol. 23, no. 15, p. 7023, 2023.

[10] J. Gonzalez *et al.*, "Implementation of a speech recognition system in a DSC," *IEEE Latin America Transactions*, vol. 14, no. 6, pp. 2657–2662, 2016.

APPENDIX

This appendix provides step-by-step instructions for setting up the complete development environment, flashing the ESP32 firmware, and building and installing the Android application.

A.1 Prerequisites

The following tools must be installed on the host PC before proceeding:

- **Python 3.8+** — required for template conversion and PlatformIO's build system. Available at https://www.python.org.
- **Git** — for cloning the repository. Available at https://git-scm.com.
- **VS Code** (recommended) or any editor with PlatformIO support.

A.2 ESP32 Firmware

1) **Install PlatformIO.** Install the PlatformIO IDE extension in VS Code, or install the PlatformIO Core CLI via:

```
pip install platformio
```

2) **Clone / extract the project.** Place the `ugv_esp32/` folder on your PC.
3) **Generate MFCC template binaries.** Run the conversion script once to produce the `.bin` files consumed by the Android app:

```
cd ugv_esp32
python convert_templates.py
```

This reads `templates.h` and writes one binary file per command into the `assets/` folder.
4) **Connect the ESP32** to the PC via USB.
5) **Build and flash** the firmware:

```
pio run --target upload
```

PlatformIO automatically installs the `ESP32 BLE Arduino` library declared in `platformio.ini`. After flashing, the ESP32 advertises itself as `ESP32_UGV` over BLE and awaits a connection.
6) **Verify** by opening the serial monitor (`pio device monitor`, 115200 baud) and confirming the message `BLE started, advertising....`

A.3 Android Application

Option A — Install the pre-built APK (recommended):

1) Copy `ugv_android.apk` to the Android device.
2) On the device, enable *Settings* → *Security* → *Install unknown apps* for your file manager.
3) Open the APK file and tap *Install*.

Option B — Build from source with Android Studio:

1) **Install Android Studio** (Hedgehog or later) from https://developer.android.com/studio. During setup, install the Android SDK (API level 33+) and the Android NDK if prompted.
2) **Open the project.** In Android Studio, choose *File* → *Open* and select the `ugv_android/` folder. Wait for Gradle sync to complete.
3) **Copy template assets.** Paste the `.bin` files generated in Section A.2 Step 3 into:

```
ugv_android/app/src/main/assets/
```

4) **Build the APK.** Select *Build* → *Build Bundle(s) / APK(s)* → *Build APK(s)*. The signed debug APK is placed in:

```
app/build/outputs/apk/debug/app-debug.apk
```

5) **Deploy to device.** Connect an Android phone (USB debugging enabled), then click *Run* → *Run 'app'* or use:

```
adb install app/build/outputs/apk/debug/app-
    debug.apk
```

A.4 First-Time Setup on the Phone

1) Enable Bluetooth on the Android device.
2) Open the **UGV Controller** app. The *SetupActivity* wizard launches automatically on first run, guiding you through BLE pairing.
3) Tap *Connect to ESP32*. The app scans for a BLE device named `ESP32_UGV` and connects automatically.
4) Follow the on-screen prompts to record 5 templates per command (10 recommended for best accuracy).
5) Tap any command button to issue voice commands. The recognized command and DTW score are shown on the main screen; detailed per-command scores are available in the *Debug* screen.

A.5 Troubleshooting

- **AudioRecord failed:** A 200 ms delay is already applied between recordings. If the error persists, close other apps using the microphone.
- **BLE not connecting:** Ensure the ESP32 is powered and advertising (`ESP32_UGV` appears in a BLE scanner app). Force-stop the UGV app and reconnect.
- **Poor recognition accuracy:** Record additional templates in a quiet environment. Reduce the threshold τ in `DTWMatcher.java` if valid commands are being rejected.
- **Gradle build errors:** Ensure JDK 17 is selected in *File* → *Project Structure* → *SDK Location*.